Bhartiya Vidya Bhavans
Sardar Patel Institute of Technology
Munshi Nagar – Andheri West – 400058

Name: Atharv patil (2024300165)
Tanish parakh (2024300157)
Gitesh patil (2024300166)
Branch: C.E

DIVISION: C
BATCH: C1
DATE: 23/08/2026

Experiment No.: 3

Aim : To extend a Java-based Multithreaded Distributed Banking Home Loan Management System by implementing Cristian's Algorithm (Client-Pull Method) and the Berkeley Algorithm (Server-Push Time Daemon), ensuring precise clock synchronization and accurate timestamping for concurrent loan applications and EMI transactions across distributed clients.

Theory :

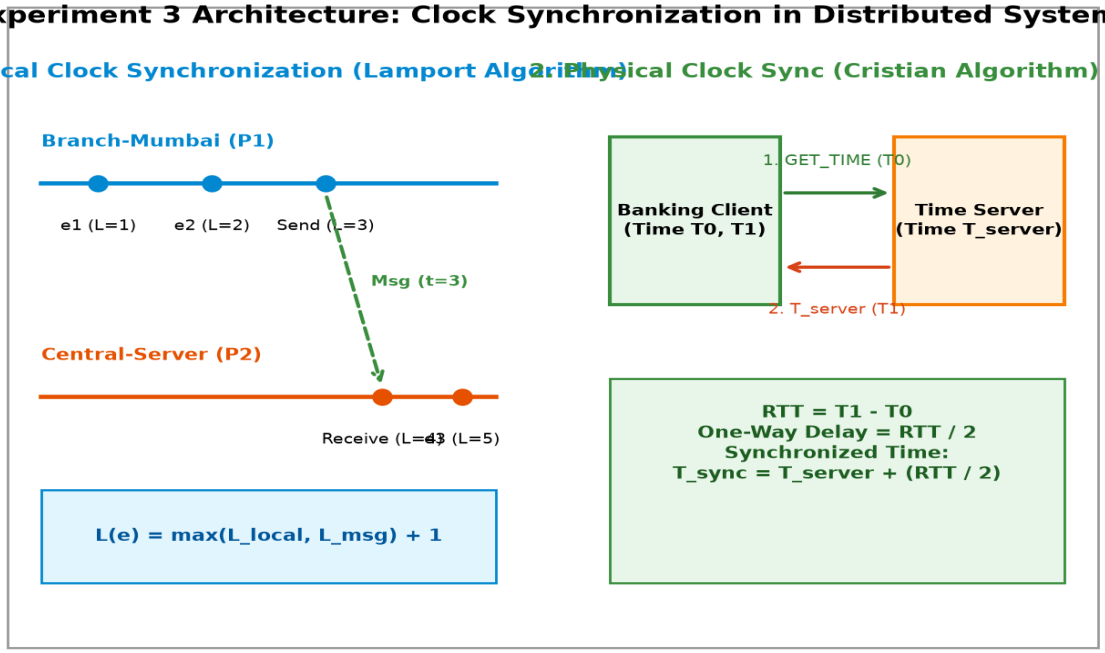
Clock Synchronization in Distributed Banking Systems is critical to align local system times across independent client nodes and servers, maintaining a globally consistent transaction ledger. In distributed banking, if physical clocks drift due to hardware oscillator variance, financial transactions (loan submissions, EMI account debits) can be logged out of order.

1. Cristian's Algorithm (Client-Pull Method):
• Concept: A Loan Client actively requests the exact time from the central Loan Server right before executing an EMI transaction.
• Implementation: The client records local time T0 before dispatching the SYNC_CRISTIAN request to the server.
• Server Response: The server receives this request and replies with its current system time T_server.
• Calculation: The client receives the response at T1, computes Round Trip Time RTT = T1 - T0, and adjusts its clock to T_sync = T_server + (RTT / 2).

2. Berkeley Algorithm (Server-Push Time Daemon Method):
• Concept: The central Loan Server acts as an active Time Daemon, ensuring all connected banking clients are synchronized.
• Implementation: A background daemon thread in the server periodically polls active client sockets with a POLL_TIME prompt.
• Calculation: Clients respond with their local timestamps (TIME_REPLY). The server calculates the average time skew and sends individual offset adjustments back so clients correct local drift.

Key Concepts Demonstrated:
● **Cristian's Algorithm:** Adjusts client clock by querying time server and compensating for RTT / 2.
● **Berkeley Algorithm:** Utilizes a central time daemon to poll clients, calculate time skew, and issue offset adjustments.
● **Clock Synchronization:** Aligns local system times across distributed banking nodes.
● **Transaction Integrity:** Guarantees concurrent home loan submissions and EMI payments are recorded with synchronized timestamps.

Problem-solving :



Code :

1. MultiThreadedLoanServer.java (Server with Cristian & Berkeley Daemon)

```
import java.io.*;
import java.net.*;
import java.util.*;

public class MultiThreadedLoanServer {

    private static final int PORT = 5000;
    private static Map<PrintWriter, Socket> clientSockets = new HashMap<>();
    private static Map<String, LoanRecord> loanDatabase = new HashMap<>();
    private static int loanCounter = 1001;

    // Helper class for loan records
    static class LoanRecord {
        String loanId;
        String customerId;
        String customerName;
        double totalAmount;
        double remainingBalance;
        int tenureYears;
        double interestRate;
        String status;

        LoanRecord(String loanId, String customerId, String customerName, double amount, int tenureYears, double
interestRate) {
            this.loanId = loanId;
            this.customerId = customerId;
            this.customerName = customerName;
            this.totalAmount = amount;
            this.remainingBalance = amount;
            this.tenureYears = tenureYears;
            this.interestRate = interestRate;
            this.status = "UNDER_REVIEW";
        }

        @Override
        public String toString() {
            return "Loan ID: " + loanId +
```

```
                    " | Customer: " + customerName +
                    " (ID: " + customerId + ")" +
                    " | Total: ■" + totalAmount +
                    " | Balance: ■" + remainingBalance +
                    " | Tenure: " + tenureYears + " yrs" +
                    " | Interest: " + interestRate + "%" +
                    " | Status: " + status;
            }
        }

        public static void main(String[] args) throws IOException {
            ServerSocket serverSocket = new ServerSocket(PORT);
            System.out.println("================================================================================");
            System.out.println(" MULTITHREADED HOME LOAN SERVER WITH CLOCK SYNC ON PORT " + PORT);
            System.out.println(" Algorithms Active: Cristian's (Client-Pull) & Berkeley (Server-Push Daemon)");
            System.out.println(" Waiting for client connections...");
            System.out.println("================================================================================");

            // ---------------------------------------------------------------------
            // BERKELEY ALGORITHM TIME DAEMON THREAD
            // Periodically polls connected clients, collects timestamps, computes average offset
            // ---------------------------------------------------------------------
            new Thread(() -> {
                try {
                    while (true) {
                        Thread.sleep(12000); // Poll interval (12 seconds)

                        synchronized (clientSockets) {
                            if (clientSockets.isEmpty()) continue;

                            long serverTime = System.currentTimeMillis();
                            System.out.println("\n[BERKELEY TIME DAEMON] Starting Time Synchronization Pulse...");
                            System.out.println("[BERKELEY TIME DAEMON] Server Time (Master): " + serverTime + " ms");

                            List<PrintWriter> activeWriters = new ArrayList<>(clientSockets.keySet());
                            System.out.println("[BERKELEY TIME DAEMON] Polling " + activeWriters.size() + " active cl
ients for local timestamps...");

                            // Broadcast POLL_TIME to active clients
                            for (PrintWriter writer : activeWriters) {
                                writer.println("POLL_TIME");
                            }
                        }
                    }
                } catch (Exception e) {
                    System.out.println("[BERKELEY ERROR] " + e.getMessage());
                }
            }).start();

            // Server socket accept loop
            while (true) {
                Socket clientSocket = serverSocket.accept();
                System.out.println("[SERVER LOG] New Client Connected: " + clientSocket.getRemoteSocketAddress());
                new Thread(new ClientHandler(clientSocket)).start();
            }
        }

        private static class ClientHandler implements Runnable {
            private Socket socket;
            private PrintWriter out;
            private BufferedReader in;

            ClientHandler(Socket socket) {
                this.socket = socket;
            }

            @Override
            public void run() {
                try {
                    in = new BufferedReader(new InputStreamReader(socket.getInputStream()));
```

```
                out = new PrintWriter(socket.getOutputStream(), true);

                synchronized (clientSockets) {
                    clientSockets.put(out, socket);
                }

                String request;
                while ((request = in.readLine()) != null) {
                    if (request.startsWith("TIME_REPLY|")) {
                        // Handle Berkeley Algorithm time reply from client
                        long clientSimulatedTime = Long.parseLong(request.split("\\|")[1]);
                        long serverTime = System.currentTimeMillis();
                        long offset = clientSimulatedTime - serverTime;
                        System.out.println("[BERKELEY DAEMON] Received TIME_REPLY from " + socket.getRemoteSocket
Address() +
                                           " -> Client Time: " + clientSimulatedTime + " ms | Calculated Offset:
" + offset + " ms");

                        // Send recommended offset correction back to client
                        long correction = -offset / 2; // Offset adjustment
                        out.println("BERKELEY_ADJUST|" + correction);
                        continue;
                    }

                    System.out.println("[THREAD " + Thread.currentThread().getId() + "] Request: " + request);
                    String response = processRequest(request);
                    out.println("RESPONSE|" + response);
                }

            } catch (IOException e) {
                System.out.println("[SERVER LOG] Client disconnected: " + socket.getRemoteSocketAddress());
            } finally {
                synchronized (clientSockets) {
                    if (out != null) {
                        clientSockets.remove(out);
                    }
                }
                try {
                    socket.close();
                } catch (IOException e) {
                    // Ignore close exception
                }
            }
        }

        private String processRequest(String request) {
            try {
                String[] data = request.split("\\|");
                String command = data[0];

                // ---------------------------------------------------------------
                // CRISTIAN'S ALGORITHM: Return current server system time in ms
                // ---------------------------------------------------------------
                if (command.equals("SYNC_CRISTIAN")) {
                    long serverTime = System.currentTimeMillis();
                    return String.valueOf(serverTime);
                }
                else if (command.equals("SUBMIT")) {
                    String customerId = data[1];
                    String customerName = data[2];
                    double amount = Double.parseDouble(data[3]);
                    int tenure = Integer.parseInt(data[4]);
                    double interest = Double.parseDouble(data[5]);

                    String loanId;
                    LoanRecord record;

                    synchronized (MultiThreadedLoanServer.class) {
                        loanId = "HL" + loanCounter++;
                        record = new LoanRecord(loanId, customerId, customerName, amount, tenure, interest);
```

```
                    loanDatabase.put(loanId, record);
                }

                broadcast("[NOTIFICATION] New Loan Application Registered: " + loanId + " by " + customerName
);
                return "Loan Application Submitted Successfully! " + record.toString();
            }
            else if (command.equals("PAY_EMI")) {
                String loanId = data[1];
                double paymentAmount = Double.parseDouble(data[2]);

                synchronized (MultiThreadedLoanServer.class) {
                    if (!loanDatabase.containsKey(loanId)) {
                        return "ERROR: Loan ID " + loanId + " not found.";
                    }
                    LoanRecord record = loanDatabase.get(loanId);
                    if (record.remainingBalance <= 0) {
                        return "Loan ID " + loanId + " is already FULLY_PAID!";
                    }

                    record.remainingBalance -= paymentAmount;
                    if (record.remainingBalance <= 0) {
                        record.remainingBalance = 0;
                        record.status = "CLOSED_FULLY_PAID";
                    } else {
                        record.status = "ACTIVE_APPROVED";
                    }
                    broadcast("[NOTIFICATION] Payment of ■" + paymentAmount + " processed for " + loanId + ".
 Remaining: ■" + record.remainingBalance);
                    return "Payment Received! Updated Record: " + record.toString();
                }
            }
            else if (command.equals("STATUS")) {
                String loanId = data[1];
                synchronized (MultiThreadedLoanServer.class) {
                    if (loanDatabase.containsKey(loanId)) {
                        LoanRecord record = loanDatabase.get(loanId);
                        return "Loan ID: " + record.loanId + " | Status: " + record.status + " | Balance: ■"
+ record.remainingBalance;
                    } else {
                        return "ERROR: Loan ID " + loanId + " not found.";
                    }
                }
            }
            else if (command.equals("DETAILS")) {
                String loanId = data[1];
                synchronized (MultiThreadedLoanServer.class) {
                    if (loanDatabase.containsKey(loanId)) {
                        return loanDatabase.get(loanId).toString();
                    } else {
                        return "ERROR: Loan ID " + loanId + " not found.";
                    }
                }
            }
            else {
                return "ERROR: Invalid command.";
            }

        } catch (Exception e) {
            return "ERROR processing request: " + e.getMessage();
        }
    }

    private void broadcast(String message) {
        synchronized (clientSockets) {
            for (PrintWriter writer : clientSockets.keySet()) {
                writer.println("BROADCAST|" + message);
            }
        }
```

```
        }
    }
}
```

2. LoanClient.java (Client with Cristian Sync & Berkeley Listener)

```
import java.io.*;
import java.net.*;
import java.text.SimpleDateFormat;
import java.util.*;

public class LoanClient {

    private static final String SERVER_HOST = "localhost";
    private static final int SERVER_PORT = 5000;

    // Simulate local clock drift (+150ms) for demonstration on localhost
    private static long simulatedDriftMs = 150;

    public static void main(String[] args) {
        try {
            Socket socket = new Socket(SERVER_HOST, SERVER_PORT);
            BufferedReader in = new BufferedReader(new InputStreamReader(socket.getInputStream()));
            PrintWriter out = new PrintWriter(socket.getOutputStream(), true);
            Scanner scanner = new Scanner(System.in);

            System.out.println("================================================================================");
            System.out.println(" MULTITHREADED HOME LOAN CLIENT WITH CLOCK SYNCHRONIZATION");
            System.out.println("================================================================================");

            // MULTITHREADED BACKGROUND LISTENER THREAD
            // Listens for server responses, real-time broadcasts, and Berkeley Algorithm POLL_TIME prompts
            Thread listenerThread = new Thread(() -> {
                try {
                    String serverMessage;
                    while ((serverMessage = in.readLine()) != null) {
                        if (serverMessage.startsWith("POLL_TIME")) {
                            // BERKELEY ALGORITHM RESPONSE
                            long simulatedLocalTime = System.currentTimeMillis() + simulatedDriftMs;
                            System.out.println("\n[BERKELEY DAEMON PULSE] Server polled local time.");
                            System.out.println("[BERKELEY DAEMON PULSE] Sending simulated local time: " + simulat
edLocalTime + " ms (Drift: +" + simulatedDriftMs + "ms)");
                            out.println("TIME_REPLY|" + simulatedLocalTime);
                            System.out.print("Enter choice: ");
                        }
                        else if (serverMessage.startsWith("BERKELEY_ADJUST|")) {
                            long adjustment = Long.parseLong(serverMessage.split("\\|")[1]);
                            simulatedDriftMs += adjustment;
                            System.out.println("\n[BERKELEY DAEMON ADJUST] Received time correction: " + adjustme
nt + " ms. New Adjusted Drift: +" + simulatedDriftMs + "ms");
                            System.out.print("Enter choice: ");
                        }
                        else if (serverMessage.startsWith("BROADCAST|")) {
                            System.out.println("\n[REAL-TIME BROADCAST] " + serverMessage.substring(10));
                            System.out.print("Enter choice: ");
                        } else if (serverMessage.startsWith("RESPONSE|")) {
                            String responseBody = serverMessage.substring(9);
                            // Check if this response is for Cristian's Algorithm
                            try {
                                long tServer = Long.parseLong(responseBody.trim());
                                // Handled in main thread Cristian sync block
                            } catch (NumberFormatException e) {
                                System.out.println("\n[SERVER RESPONSE] " + responseBody);
                            }
                        } else {
                            System.out.println("\n[SERVER MESSAGE] " + serverMessage);
                        }
                    }
                } catch (IOException e) {
                    System.out.println("\n[DISCONNECTED] Server connection closed.");
                }
            });
```

```
            listenerThread.setDaemon(true);
            listenerThread.start();

            // Main thread: CLI menu and request handler
            while (true) {
                printMenu();
                String choice = scanner.nextLine().trim();

                if (choice.equals("1")) {
                    System.out.print("Customer ID: ");
                    String customerId = scanner.nextLine().trim();

                    System.out.print("Customer Name: ");
                    String customerName = scanner.nextLine().trim();

                    System.out.print("Loan Amount (■): ");
                    double amount = Double.parseDouble(scanner.nextLine().trim());

                    System.out.print("Loan Tenure (years): ");
                    int tenure = Integer.parseInt(scanner.nextLine().trim());

                    System.out.print("Interest Rate (%): ");
                    double interest = Double.parseDouble(scanner.nextLine().trim());

                    String request = "SUBMIT|" + customerId + "|" + customerName + "|" + amount + "|" + tenure +
"|" + interest;
                    out.println(request);
                    Thread.sleep(300);

                } else if (choice.equals("2")) {
                    System.out.print("Enter Loan ID: ");
                    String loanId = scanner.nextLine().trim();

                    System.out.print("Enter Payment / EMI Amount (■): ");
                    double amount = Double.parseDouble(scanner.nextLine().trim());

                    String request = "PAY_EMI|" + loanId + "|" + amount;
                    out.println(request);
                    Thread.sleep(300);

                } else if (choice.equals("3")) {
                    System.out.print("Enter Loan ID: ");
                    String loanId = scanner.nextLine().trim();

                    String request = "STATUS|" + loanId;
                    out.println(request);
                    Thread.sleep(300);

                } else if (choice.equals("4")) {
                    System.out.print("Enter Loan ID: ");
                    String loanId = scanner.nextLine().trim();

                    String request = "DETAILS|" + loanId;
                    out.println(request);
                    Thread.sleep(300);

                } else if (choice.equals("5")) {
                    // ---------------------------------------------------------------
                    // CRISTIAN'S ALGORITHM: Client-Pull Clock Synchronization
                    // ---------------------------------------------------------------
                    long t0 = System.currentTimeMillis(); // T0: Send time
                    out.println("SYNC_CRISTIAN");

                    String serverResponse = in.readLine();
                    long t1 = System.currentTimeMillis(); // T1: Receive time

                    if (serverResponse != null && serverResponse.startsWith("RESPONSE|")) {
                        long tServer = Long.parseLong(serverResponse.substring(9).trim());
                        long rtt = t1 - t0;
                        long synchronizedTime = tServer + (rtt / 2);
```

```
                        SimpleDateFormat sdf = new SimpleDateFormat("HH:mm:ss.SSS");
                        System.out.println("\n-------------------------------------------------------------------------
-------");
                        System.out.println(" CRISTIAN'S ALGORITHM CLOCK SYNCHRONIZATION RESULTS");
                        System.out.println("-------------------------------------------------------------------------
-----");
                        System.out.println("T0 (Client Send Time)          : " + t0 + " ms (" + sdf.format(new Da
te(t0)) + ")");
                        System.out.println("T_server (Server System Time)   : " + tServer + " ms (" + sdf.format(
new Date(tServer)) + ")");
                        System.out.println("T1 (Client Receive Time)       : " + t1 + " ms (" + sdf.format(new Da
te(t1)) + ")");
                        System.out.println("Round Trip Time (RTT)          : " + rtt + " ms");
                        System.out.println("Network One-Way Delay (RTT / 2): " + (rtt / 2.0) + " ms");
                        System.out.println("SYNCHRONIZED PHYSICAL TIME     : " + synchronizedTime + " ms (" + sdf
.format(new Date(synchronizedTime)) + ")");
                        System.out.println("-------------------------------------------------------------------------
-----");
                    }
                    Thread.sleep(300);

                } else if (choice.equals("6")) {
                    System.out.println("Exiting Distributed Home Loan Client...");
                    socket.close();
                    break;
                } else {
                    System.out.println("Invalid choice. Please enter 1-6.");
                }
            }

            scanner.close();

        } catch (Exception e) {
            System.err.println("Error connecting to server: " + e.getMessage());
        }
    }

    private static void printMenu() {
        System.out.println("\n-------------------------------------------------");
        System.out.println("1. Apply for Home Loan (SUBMIT)");
        System.out.println("2. Make EMI / Account Payment (PAY_EMI)");
        System.out.println("3. Check Loan Status (STATUS)");
        System.out.println("4. View Loan Details (DETAILS)");
        System.out.println("5. Synchronize Physical Clock (Cristian's Algorithm)");
        System.out.println("6. Exit");
        System.out.print("Enter choice: ");
    }
}
```

Output screenshot :

Figure 1: Multithreaded Home Loan Server Started with Berkeley Time Daemon

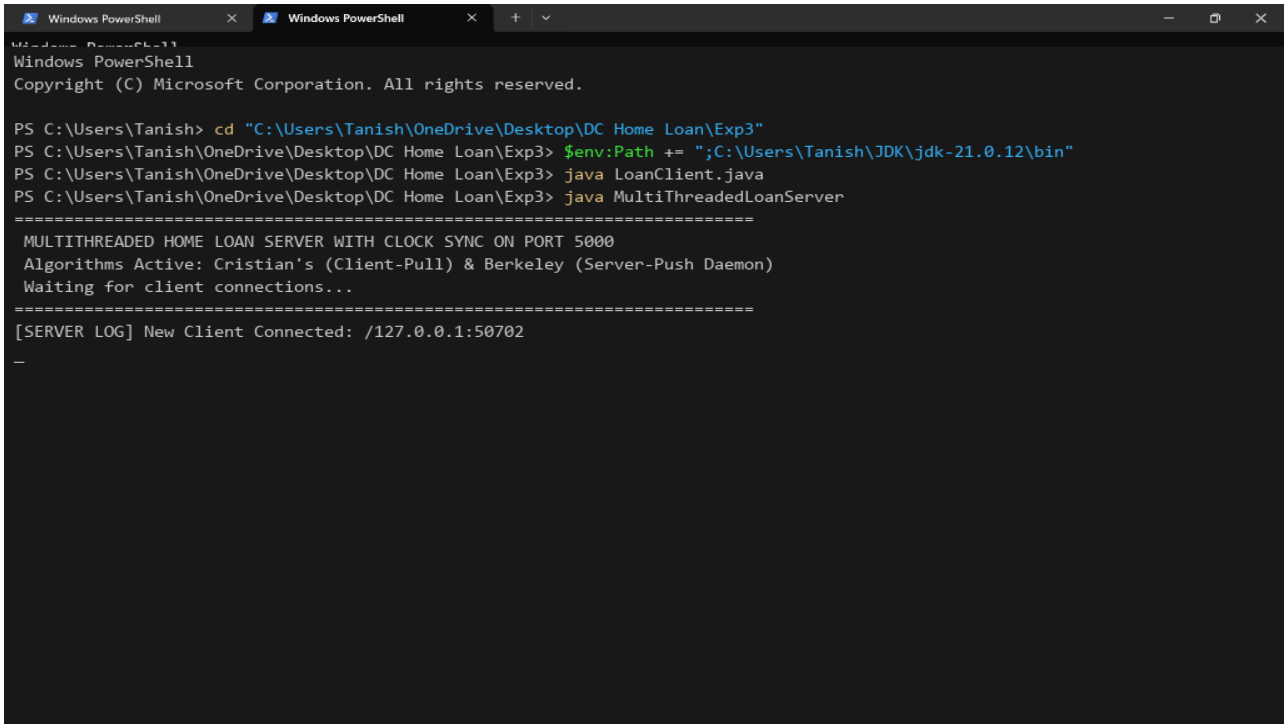


Figure 2: Client Executing Cristian's Algorithm Clock Synchronization (SYNC_CRISTIAN)

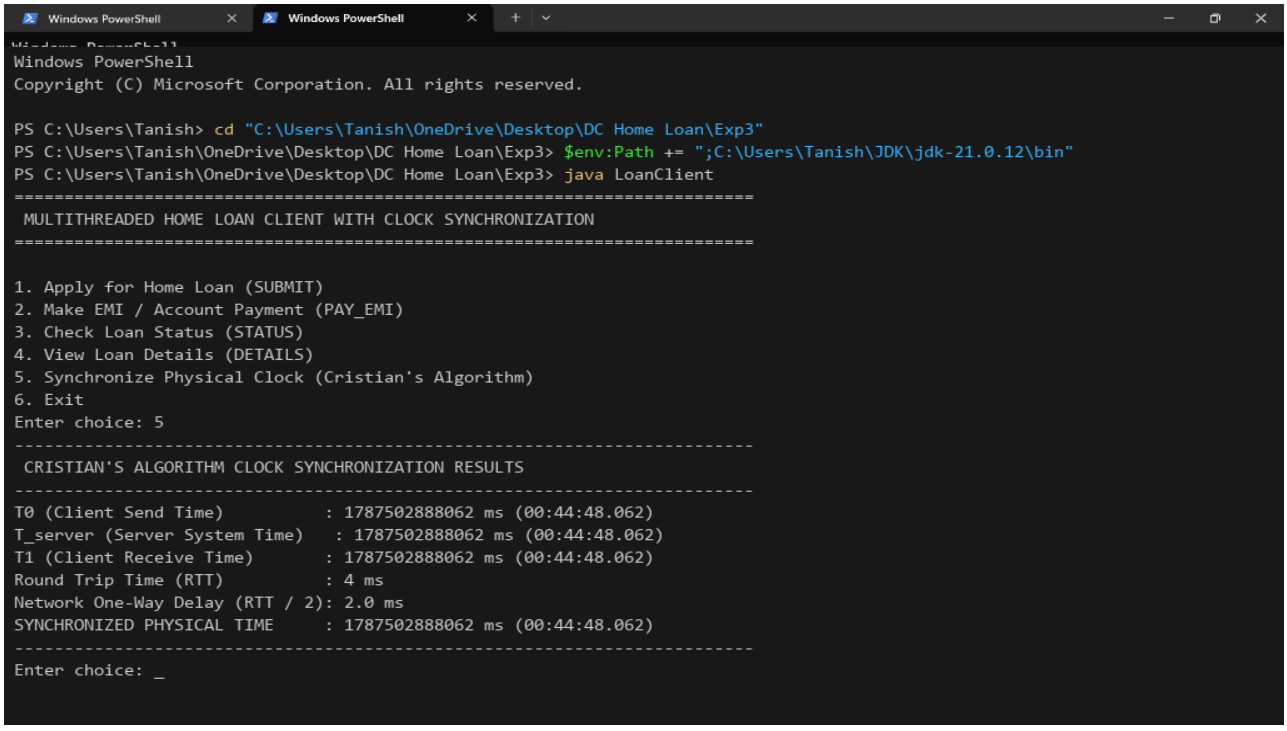


Figure 3: Client Receiving Berkeley Time Daemon Pulse and Applying Offset Adjustment

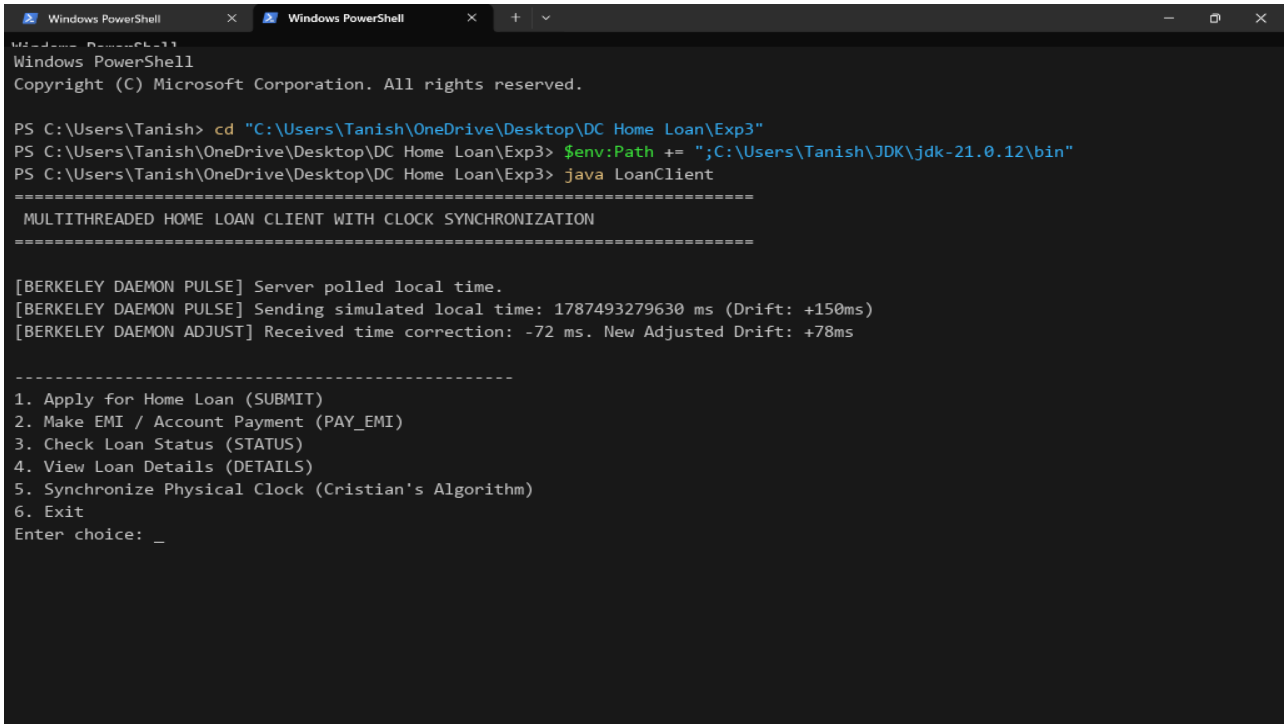
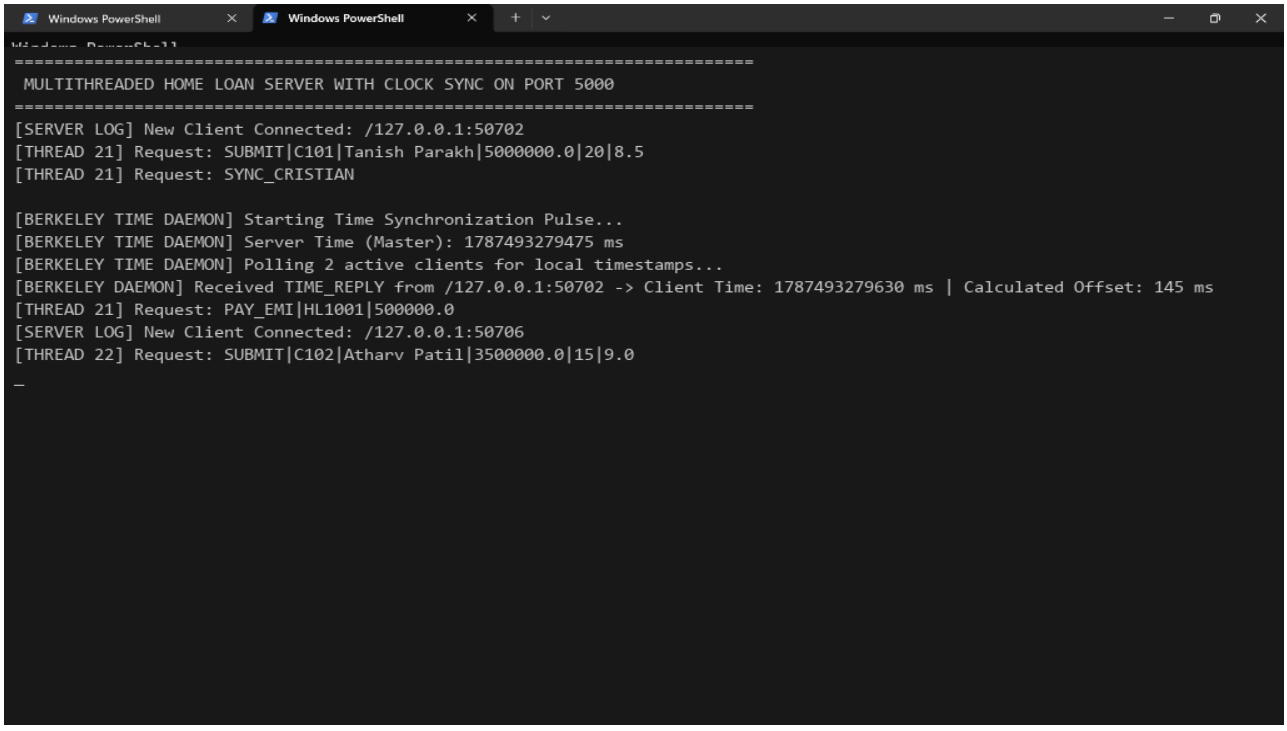


Figure 4: Multithreaded Server Execution Log Showing Berkeley Daemon Polling & Time Adjustments



Conclusion: (It should include what the student has learned/ understood from the experiment).

In this experiment, we successfully extended our multithreaded distributed banking home loan system by implementing both Cristian's Algorithm (Client-Pull Method) and the Berkeley Algorithm (Server-Push Time Daemon). We demonstrated how Cristian's algorithm enables a client to query server time and adjust its local clock using RTT / 2 compensation. Furthermore, we showed how the Berkeley Time Daemon actively polls connected clients, computes average time skew, and broadcasts offset adjustments to correct local clock drift. This experiment effectively highlighted how clock synchronization guarantees transaction integrity in distributed banking software.

Attach research paper based on experiment performed into the classroom (Not for all the practicals)

(Students are required to prepare one doc file for each experiment and then upload it in pdf form.)